

Experiment 10 — Third sweep: eight real-application pipelines, three systems

Mathilda — execution-speed experiments

Contents

Experiment 10 — Third sweep: eight real-application pipelines, three systems	1
Two hypotheses	1
The eight kernels	1
Seven fixes, six of them the same bug	2
Results	2
Three things that were wrong about the work itself	3
Two doc claims and two tests that had gone stale	3
Verification	3
Why Mathilda is not the fastest here, and what it would take	3

Experiment 10 — Third sweep: eight real-application pipelines, three systems

Date: 2026-07-31 · Harness: [comparisons/hpc_bench.py](#) · Report: [docs/design/performance.md](#) §9 ·
Result: N-body 55.5 s → 439 ms; k-means and logistic regression went from not finishing to finishing
Common method in [README.md](#).

Two hypotheses

1. A composition can be slow in ways none of its parts are. The second sweep's conjugate-gradient row was slower than Wolfram while every primitive in it was at or ahead of parity. Primitive benchmarks cannot find that, so this sweep is built entirely out of pipelines from real applications.
2. Two systems is not enough. "Mathilda is 1.2× Mathematica" answers whether we are behind a competitor. It does not answer whether we are behind the machine. NumPy on this host links the same Accelerate BLAS Mathilda does, so on the dense rows all three run identical kernels and any spread is pure overhead.

That second hypothesis paid immediately. Three rows read acceptably against Mathematica and badly against NumPy — the sieve is 1.20× ahead of Mathematica and 26.9× behind NumPy; the return-series kernel was 13.5× behind one and 66.5× behind the other. Only the third column separates "slower than a competitor" from "slower than a memory copy".

The eight kernels

id	kernel	field	what it stresses
bsmc	Black–Scholes Monte Carlo, 10^7 paths	finance	RNG at scale, Box–Muller, reduction
tseries	EMA + rolling vol + max drawdown, 10^6	quant finance	sequential scans

id	kernel	field	what it stresses
logreg	Logistic regression, 200000×32, 100 GD steps	ML	matrix–vector both ways, loop-carried state
kmeans	k-means, 100000 points, 8 dims, k = 16, 20 its	ML	masked reductions, argmin, scatter-as-dot
nbody	All-pairs gravity, 1024 bodies, 10 Verlet steps	astrophysics	Outer, megabyte intermediates
heat3d	3D 7-point stencil, 128 ³ , 50 steps	CFD	rank-3 shifts, 16 MB working set
psd	Welch PSD, 2 ²² samples, 1024 blocks of 4096	signal processing	many medium FFTs, windowing
imgpipe	Separable Gaussian blur + Sobel, 1024 ²	vision	rank-2 ListCorrelate, Transpose

Every kernel has a deterministic scalar check all three systems agree on. That is a stronger constraint than it sounds: the check pins the algorithm, so the three columns cannot silently be timing three different computations. None disagreed.

Two of the eight did not finish on the first run.

Seven fixes, six of them the same bug

The packing decision is made about one value in isolation, but the cost is paid in proportion to another.

PACK_MIN_ELEMENTS is 250 for blast-radius reasons, which is right. What was wrong is that a 32-element vector, correctly left unpacked, could drag a 6.4-million-element matrix onto the symbolic path with it.

what	measured	after
Dot, one plain operand (20000×40 by 40)	320 ms	0.31 ms
Total[m, {1}] vs the identical Total[m]	258 ms	1.28 ms
Outer[Subtract, v, v], 1024 ²	808 ms	4.03 ms
rank-2 ⊖ rank-1 threading, 8×100000	322 ms	2.52 ms
a + b, one plain operand, 10 ⁶	418 ms	50.5 ms
the N-body step re-applied to its own output	5.75 s	43 ms
MapThread[Min, ...], 16×100000	365 ms	3.87 ms

The rule, now applied at Dot, at Plus/Times, and at the evaluator's Listable gate:

Pack the small operand up; never materialise the large one down.

Packing is value-preserving by contract, so there is nothing to weigh.

The largest win was a return statement

Not a kernel. {px, py, pz, vx, vy, vz} came back from the N-body step as six plain Lists, because a List built out of packed rows materialised every one of them. So a function returning several arrays destroyed them all on the way out, and the caller paid on the next operation: 42 ms for the first call, 5.75 s for the second.

Profiling the slow call pointed at Outer and the elementwise arithmetic. All of them were innocent. The cost had been incurred one call earlier, by a return.

List cannot be made packed-aware — a List node holding EXPR_NDARRAY elements is exactly the malformed shape the gate exists to prevent. The fix is to pack_offer the whole node in evaluate_step, because pack_sniff already absorbs already-packed rows: n packed vectors are a rank-2 buffer.

Results

Benchmark	before	after	Mathematica 14.0	NumPy 2.4.4	
N-body all-pairs gravity, 1024 bodies	55.5 s	439 ms	6.98 s	396 ms	15.9× faster than V
k-means, 100000×8, k=16, 20 its	unbounded	2.26 s	7.33 s	765 ms	3.24× faster than V

Benchmark	before	after	Mathematica 14.0	NumPy 2.4.4	
Logistic regression, 200000×32, 100 steps	unbounded	2.71 s	2.15 s	839 ms	
Black–Scholes MC, 10 ⁷ paths	—	565 ms	349 ms	560 ms	at NumPy parity
3D heat equation, 128 ³ , 50 steps	—	3.63 s	2.55 s	2.25 s	
Welch PSD, 2 ²² samples	—	150 ms	90.2 ms	93.9 ms	
Return series, 10 ⁶	—	2.02 s	149 ms	30.4 ms	
Gaussian blur + Sobel, 1024 ²	—	2.19 s	97.0 ms	113 ms	

51 benchmarks, three systems, one run, no value mismatches.

The last two rows are the ones that made experiment 11 necessary: acceptable against nothing, and 66× and 19× behind NumPy.

Three things that were wrong about the work itself

My first **Outer** fast path did nothing. It was written, it was correct, it was verified against the List path — and it never ran, because Outer was not on `pack.c`'s aware list, so the gate had already turned both operands into Lists before `builtin_outer` saw them. Third time that exact sequence has happened (Nest, Fourier, Outer), and exactly what `make check-packed-aware` is for.

My first **ndarray_elementwise** broadcast also did nothing, for the mirror reason: the operand that needed lifting was a plain List below the threshold, so the function returned NULL before reaching the new code. Correct, verified, and worthless until the packing pre-pass was added.

My k-means and N-body kernels were wrong on the first draft. `matrix - vector` does not broadcast NumPy-style in either CAS: Plus is Listable and threads the outer level, so the smaller operand's dims must be a prefix of the larger's. I assumed rather than checked. Fixed by probing Wolfram before writing the kernel, not after.

Two doc claims and two tests that had gone stale

`packed-arrays.md` said "packing does not nest". After the List-absorption change that was false. Both the claim and the two tests asserting it still passed, because the tests used lists below the packing threshold — so the path they described was never exercised.

A test written below the threshold tests nothing.

Rewritten with above-threshold arms. This is the first of three appearances of the same lesson; see [HPC_SWEEP_3_NUMPY_GAP.md](#).

Verification

- Full 395-binary suite, clean apart from two pre-existing issues confirmed by stashing `src/` and rebuilding: one `simplify_tests` symbolic-radical case, and `primenu/moebiusmu`, which fail ~1 run in 20 on the unmodified baseline because ECM factorisation is randomised.
- `packed_list_tests` gained `test_third_sweep_fast_paths` — ~60 differential cases over all seven paths, each the packed form against the identical plain one, including every form the fast paths must decline (symbolic, Rational, int64, ragged, length mismatch, Power with a rank mismatch).
- `make check-c99`, `make check-packed-aware`, `bench_pack` all pass.
- Valgrind byte-identical to the startup baseline.

Why Mathilda is not the fastest here, and what it would take

After this sweep's seven fixes, two rows led Mathematica by 3–16× and two were still far behind NumPy. Both of the latter were closed by the sweep that followed, and the rest by the fifth:

row	at the time	now	roadmap
Return series (EMA, vol, drawdown)	66.5× NumPy	closed — 2.78× ahead of Mathematica	11-hpc-sweep-numpy-
Gaussian blur + Sobel	19× NumPy	closed — 1.10× ahead of NumPy	11-hpc-sweep-numpy-
k-means	2.92× NumPy	unfused composition	plan 9.2
Logistic regression	3.06× NumPy	a copying Transpose	17-neural-network-t
N-body, 3-D heat, Welch PSD	1.4–2.3× NumPy	unfused composition	plan 9.2

Everything remaining in this experiment is one item: interpreter-level fusion. `Compile[]` fuses (experiment 2) and ordinary array code does not, so every one of these pipelines makes three to nine passes over its data where a fused kernel would make one or two. That is the 2–3× band all five outstanding rows sit in, and it is plan item 9.2.

The logistic-regression row additionally wants the transposed view of experiment 17's roadmap, which is the same feature as experiment 14's sliding window.